

When in Doubt: Bootstrapping Reinforcement Learning Agents via Selective Vision-Language Model Guidance

Anonymous Authors¹

Abstract

Reinforcement Learning (RL) often struggles with sample inefficiency and lacks the reasoning capabilities inherent in Large Foundation Models; conversely, Vision-Language Models (VLMs) possess vast prior knowledge but suffer from high inference costs. We propose SAGE (Selective Agent Guidance via Entropy), a framework that bootstraps a lightweight RL agent using selective VLM guidance. Rather than relying on the VLM at every timestep, the agent monitors its own uncertainty during training, measured by the RL policy’s entropy as a lightweight proxy, and defers to the VLM only when it is unsure about the next action. We further employ an auxiliary behavior cloning objective to incentivize the agent to explicitly imitate the VLM’s action choices, allowing the agent to internalize valid guidance into the policy while eventually regaining full autonomy to override flawed suggestions. On classic grid-world environments and visual reasoning tasks, including a novel card-based logical environment, SAGE can outperform both the underlying RL algorithm and the VLM, with guidance being prevalent in the initial stage of training, but being naturally reduced as the policy improves. We further show that our approach outperforms multiple competitive VLM-based methods, achieving better performance with improved sample efficiency.

1. Introduction

While Reinforcement Learning (RL; Sutton et al., 1998) allows agents to master specific tasks through trial-and-error, it effectively requires learning all physics and logic from scratch, without prior information about the environment.

¹Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

This creates a bottleneck for sample efficiency, which is particularly problematic for sparse reward tasks, where much of the training budget is spent exploring to find the goal (Burda et al., 2019).

On the other hand, Large Foundation Models (LFMs; Bommasani et al., 2021) such as Large Language Models (LLMs; Yang et al., 2025) and Vision-Language Models (VLMs; Liu et al., 2023; Gemma Team et al., 2025) already provide such prior common-sense knowledge, thanks to their internet-scale pre-training corpus. As such, recent works have attempted to use LFMs directly as agents (Ahn et al., 2022; Zhai et al., 2024), but this faces two critical bottlenecks: (1) querying a multi-billion parameter model at every timestep step is computationally expensive for real-time interaction; and (2) pre-trained models do not inherently learn from environmental interaction, failing to adapt to specific task dynamics and repeating errors indefinitely.

We see the strengths and weaknesses of these two paradigms as complementary, and as such we draw inspiration from dual-process theories of cognition (Kahneman, 2011), which distinguish between a fast, habitual decision-making system and a slow, deliberative reasoning system. We thus introduce SAGE (Selective Agent Guidance via Entropy), a hybrid RL method that leverages guidance from a pre-trained VLM to bootstraps a model-free RL policy. Instead of relying on the VLM continuously, SAGE monitors the agent’s policy entropy as a proxy for uncertainty. When this exceeds a threshold, the VLM is prompted to select an action, which is then chosen by the agent. This serves a dual purpose: it provides an immediate solution to bypass exploration bottlenecks, and it creates a target for an auxiliary online Behavior Cloning (BC) objective that bootstraps the policy, as shown in Figure 1, enabling the policy to internalize the teacher’s strategy rather than just relying on it.

We evaluate SAGE on a diverse range of environments, all characterized by sparse rewards and challenging exploration, chosen to test distinct capabilities: classic grid-based navigation with FrozenLake; reasoning puzzles which require arithmetic and logical constraint satisfaction, including EZ-Points (Zhai et al., 2024), and CardMaze, a novel environment introduced in this work. We further run experiments on ALFWorld (Shridhar et al., 2021), a household simulation

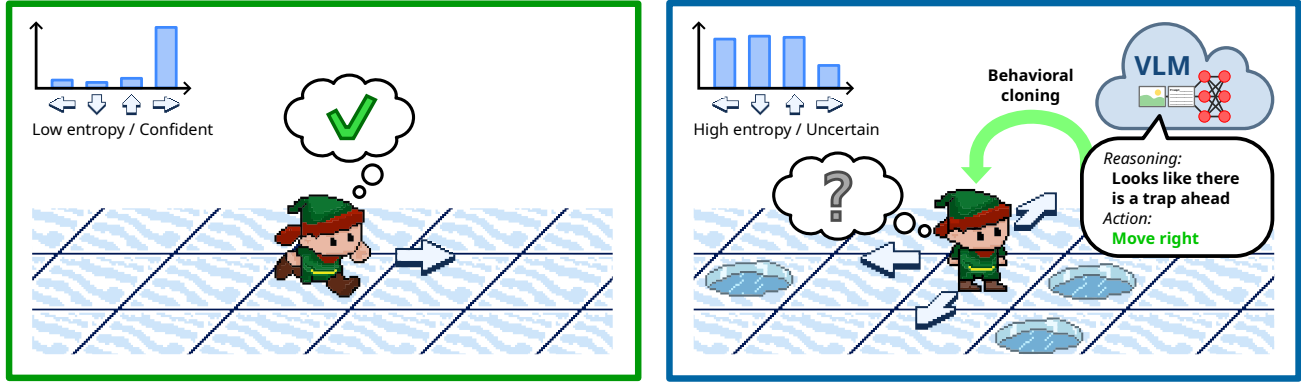


Figure 1. **Dual architecture for RL.** The agent is free to act in situations where it displays high confidence, but requests guidance from a VLM when it is uncertain, following the suggested action.

environment, to test on a realistic, high-dimensional settings, under a reduced interaction budget due to computational and time costs.

Under a sample-efficient budget of 100k environment steps, SAGE consistently outperforms competitive baselines that use VLMs for reward shaping (Wang et al., 2024), and policy distillation (Lee et al., 2025), while also exceeding the performance of both the standalone VLM policy and the unguided RL agent, highlighting the synergy between the two systems. We further report evaluation results obtained running the RL policy without any guidance on unseen tasks, and show that our method greatly outperform all baselines under this setting, demonstrating its ability to effectively internalize guidance, and eventually perform well without the need of the teacher.

Finally, we complement our results with experiments using a rule-based oracle to provide guidance, showing an upper-bound on performance and the gap caused by imperfect VLM suggestions. We also provide a sensitivity study for SAGE’s hyperparameters, and show that VLM guidance is crucial in early training to overcome exploration bottlenecks and naturally vanishes as the policy becomes confident.

2. Related Work

Imitation Learning. Imitation learning from expert policies has long been used to speed up RL (Schaal, 1996; Abbeel & Ng, 2004; Ross et al., 2011), mostly as a form of pre-training to avoid having to learn from scratch. Experts can take the form of human demonstrations (Knox & Stone, 2009) or pre-trained policies (Rusu et al., 2015; Parisotto et al., 2016), with RCMP (Da Silva et al., 2020) also using uncertainty (as measured by the standard deviation of multiple value heads) to query a stronger agent. Lately, imitation from LFM has also been considered, with LM4TEACH (Zhou et al., 2024) using an LLM expert, treating the token logits for each action as a distribution, which

is then aligned with the RL policy. Similarly, LVLM2P (Lee et al., 2025) instead uses a VLM to generate action probabilities with text. By contrast, our method uses a VLM to directly output an action, which is a more natural task, and the model is only prompted in states with high policy entropy. Finally, concurrent to our work, Merler et al. (2025) introduce a system where a VLM provides guidance under uncertainty, but their approach lacks the behavior cloning loss and the experiments are limited to a single environment.

Reward Shaping. Another approach to mitigate sample efficiency in sparse reward environments is reward shaping (Ng et al., 1999), where an additional reward function is provided to the agent, besides the one coming from the environment. Traditionally, these shaping functions were manually engineered based on domain-specific heuristics (Laud, 2004), a process that is often tedious and prone to error. Instead, LFM can be prompted to output preference labels (Du et al., 2023), scalar rewards (Kwon et al., 2023; Ye et al., 2024), code reward functions (Ma et al., 2023b; Venuto et al., 2024), used to compare the embedding distance between current and goal states (Klissarov et al., 2023; Rocamonde et al., 2024), or pre-trained as dedicated reward models (Ma et al., 2022; 2023a). Most recently, RL-VLM-F (Wang et al., 2024) proposes a framework where a VLM is occasionally prompted during training, to label a batch of pairs of transitions with a preference label, from which a reward model is trained (i.e., analogous to RL from Human Feedback for LLM training (Lambert, 2025), but with VLM-generated preferences).

Foundation Models as Policies. Finally, LFM can control an agent’s behavior directly by acting as policies (Cao et al., 2024). This can be done either with direct prompting with the current state to output either an action (Ahn et al., 2022; Huang et al., 2022a;b; Ye et al., 2024) or a code policy (Liang et al., 2023), or by adding actions as a separate dedicated modality, creating a Vision-Language-

Action Model (VLA; Brohan et al., 2023; Kim et al., 2024). Furthermore, fine-tuning LLMs with experience from the environment (effectively treating them as a RL policy simply initialized from pre-trained weights) has proven effective (Carta et al., 2023; Tan et al., 2024), with Bonetta et al. (2024) extending this approach to VLMs. Most recently, RL4VLM (Zhai et al., 2024) further develops this approach by allowing VLMs to generate a Chain of Thought (CoT; Wei et al., 2022; Kojima et al., 2022), trained with PPO (Schulman et al., 2017), allowing the resulting agent to reach good performance in environments requiring high-level reasoning. In comparison, our method uses the RL policy as the primary decision-maker, avoiding the issue of prompting the LFM at each control step or even training it, while still taking advantage of a VLM’s reasoning capabilities through behavior cloning.

3. Method

3.1. Preliminaries

We adopt the standard RL formalism, modeling the environment as a Markov Decision Process (MDP) defined by the tuple $\langle \mathcal{S}, \mathcal{A}, R, T, \gamma, \rho_0 \rangle$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $R: \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ is the reward function, $T: \mathcal{S} \times \mathcal{A} \rightarrow \mathcal{S}$ is the transition function, $\gamma \in [0, 1]$ is the discount factor and $\rho_0(\cdot) \in \Delta(\mathcal{S})$ is the initial state distribution. At each timestep t , the agent observes $s_t \in \mathcal{S}$ and must select an action $a_t \in \mathcal{A}$ from a learned parametric stochastic policy $\pi_\theta(a_t | s_t)$, trained with PPO (Schulman et al., 2017). The agent perceives states s_t as RGB images. As such, π_θ is implemented as a Convolutional Neural Network (CNN), with two separate linear layers on top acting as the policy and value heads. We focus this study on environments with a discrete action space $\mathcal{A}$, and deterministic transition function T , in order to simplify the problem and test SAGE’s effectiveness under controlled conditions. However, the method is generally applicable with minimal changes.

Our objective is to combine this lightweight model-free policy with an external more expensive **teacher policy** $\pi^T(\cdot | s_t)$, such as a VLM, in a way that is selective, sample efficient, and allows the RL policy to progressively internalize the teacher’s behavior, while only requesting guidance when it is uncertain to avoid unnecessary calls.

3.2. Entropy-Gated Selective Guidance

We consider the **entropy** H_t of the policy $\pi_\theta(\cdot | s_t)$ as a proxy to measure the agent’s uncertainty in a given state:

$$H_t = H[\pi_\theta(\cdot | s_t)] = - \sum_{a \in \mathcal{A}} \pi_\theta(a | s_t) \log \pi_\theta(a | s_t), \quad (1)$$

which is normalized as $\hat{H}_t = \frac{H_t}{\log |\mathcal{A}|}$, so that $\hat{H}_t \in [0, 1]$ for any discrete action space.

Algorithm 1 SAGE Data Collection Loop

Require: Policy π_θ , teacher policy π^T , entropy threshold ν , guidance probability κ
Require: VLM cache C , rollout buffer $\mathcal{B}$

- 1: Observe state s_t
- 2: Compute normalized policy entropy $\hat{H}_t$
- 3: **if** $\hat{H}_t > \nu$ **and** guidance is triggered with probability κ **then**
- 4: Obtain teacher action a_t (from cache C if available, otherwise from π^T)
- 5: Mark transition as guided: $g_t \leftarrow 1$
- 6: **else**
- 7: Sample $a_t \sim \pi_\theta(\cdot | s_t)$
- 8: $g_t \leftarrow 0$
- 9: **end if**
- 10: Execute a_t , observe (r_t, s_{t+1}, d_t)
- 11: Store $(s_t, a_t, r_t, s_{t+1}, d_t, g_t)$ in rollout buffer $\mathcal{B}$

The policy is considered to be uncertain when the entropy exceeds a fixed **threshold** $\nu \in [0, 1]$, i.e. $\hat{H}_t > \nu$. We further add a **stochastic gate**, controlled by a guidance probability $\kappa \in [0, 1]$, to randomly avoid requesting guidance in uncertain states, in order to still allow for exploration outside of the teacher’s policy. Moreover, we hypothesize that this can prevent the agent from getting trapped in circular behavior when the VLM repeatedly provides incorrect guidance. Thus, we formalize SAGE’s policy μ_θ as:

$$\mu_\theta(\cdot | s_t) \rightarrow \begin{cases} a_t^{\text{guid}} \sim \pi^T(\cdot | s_t) & \text{if } \hat{H}_t > \nu \wedge u_t < \kappa \\ a_t \sim \pi_\theta(\cdot | s_t) & \text{otherwise,} \end{cases} \quad (2)$$

where $u_t \sim \mathcal{U}(0, 1)$ is sampled from a uniform distribution. As is common in guided RL settings, this breaks the on-policy assumption of PPO; however, our goal is not unbiased policy-gradient estimation, but empirical improvement under guided exploration. In SAGE, guidance is naturally brief and applied only during early training, which further attenuates the impact of this mismatch.

Since querying a VLM is expensive, we maintain a **cache** C that stores state-action pairs returned by the model. If a state s_t matches a cache key exactly the cached action is reused; otherwise, the VLM is prompted and the new pair is stored into C . Our method is summarized in Algorithm 1.

3.3. Behavior Cloning-Augmented Guidance

Teacher-guided actions are inherently off-policy. While standard PPO can correct for distribution shifts via importance sampling, allowing it to process experience generated by slightly different policies, this fails for the large discrepancies introduced by external guidance. When the teacher

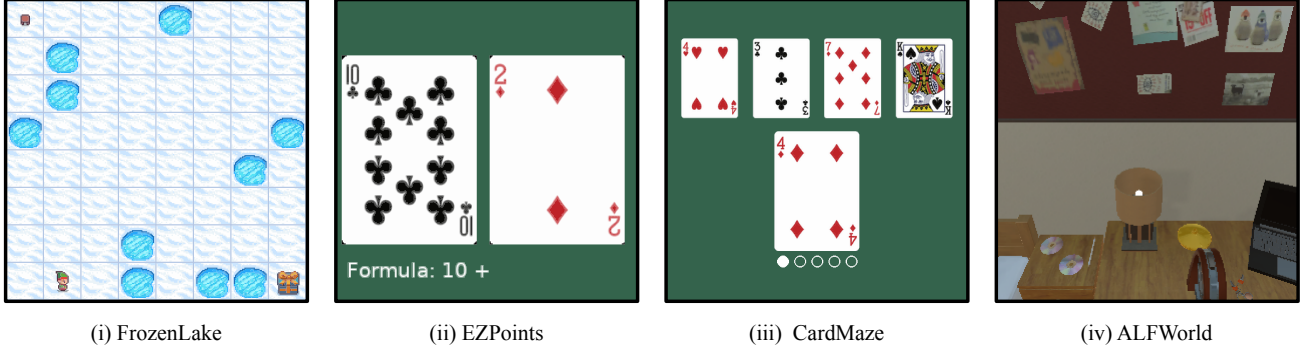


Figure 2. RL environments used in our experiments. (i) FrozenLake, (ii) EZPoints, (iii) CardMaze, (iv) ALFWorld.

forces an action that the agent considers unlikely, the resulting importance ratio becomes high and is aggressively clipped, reducing the gradient signal, and preventing the agent from internalizing guidance via the RL objective.

To strengthen the influence of guidance while retaining the PPO optimization framework, we augment the policy update with an auxiliary **behavior cloning objective** applied at guided timesteps. During data collection, each transition is annotated with a binary indicator $g_t \in \{0, 1\}$ denoting whether the action was provided by the teacher. The behavior cloning loss is then defined as:

$$\mathcal{L}_{BC}(\theta) = -\mathbb{E}_{t \in \mathcal{B}} [g_t \log \pi_\theta(a_t | s_t)]. \quad (3)$$

By including the indicator g_t inside the expectation, this objective contributes to the gradient only for transitions where guidance was requested, explicitly encouraging the policy to assign higher probability to these demonstrated actions.

Let $\mathcal{L}_{PPO}(\theta)$ denote the standard PPO clipped objective, as described by Schulman et al. (2017), and let $\beta > 0$ be a coefficient controlling the strength of behavior cloning. The total loss for the policy parameters becomes:

$$\mathcal{L}_{SAGE}(\theta) = \mathcal{L}_{PPO}(\theta) + \beta \mathcal{L}_{BC}(\theta). \quad (4)$$

The value function loss is included in $\mathcal{L}_{PPO}(\theta)$ in the standard manner. At guided timesteps, we omit entropy regularization, as behavior cloning encourages the policy to concentrate probability mass on a specific teacher-provided action, while entropy promotes action diversity. As guidance corresponds to externally provided decisions rather than agent-driven exploration, applying entropy regularization would be misaligned with the learning objective.

The behavior cloning term thus acts as a targeted bias that amplifies the impact of guidance in critical states, mitigating the effect of clipping while still allowing the policy to refine or override imperfect guidance through RL. In practice, we linearly anneal β over training, so that the policy is bootstrapped by the VLM, but increasingly relies on its own experience as learning progresses.

4. Experimental Setup

4.1. Environments

Sample images representing a state s_t for all environments are shown in Figure 2.

FrozenLake 8x8. We use a deterministic variant of the FrozenLake grid-world from Gymnasium (Towers et al., 2024), augmented with a new 8x8 map procedurally generated at every episode. The agent moves in the four cardinal directions (without slipping in random directions, as in the standard implementation) and must reach the goal while avoiding holes. The sparse reward is +1 only upon reaching the goal and 0 otherwise. Because each episode uses a different layout, the agent is unable to memorize a fixed map. This requires precise understanding of scene elements, especially holes, which is not trivial even for advanced VLMs.

EZPoints. In EZPoints (Zhai et al., 2024) the agent must construct a formula that evaluates to 12 using the numbers shown on two poker cards. The action space includes natural numbers in $[1, 10]$ and the operators $\{+, *, =\}$. Only symbols corresponding to the available cards¹ and valid operators are admissible at each step. If the action is legal, its textual representation is appended to the evolving formula, which is shown in the image; otherwise the state remains unchanged. The agent succeeds only if the final expression uses each card value exactly once and evaluates to 12, receiving reward +10. Illegal actions result in a reward of -1 (without ending the episode), everything else results in a reward of 0. Episodes terminate when the formula length exceeds 5, or when the = operator is selected. EZPoints tests the agent abilities to understand simple math and to plan logically building the correct formula without errors.

CardMaze. We introduce CardMaze as a novel perceptual symbol matching task, designed to be challenging for PPO but intuitive for a VLM. Each observation shows four candidate cards at the top, a prompt card at the bottom, and a counter of consecutive correct selections. At every timestep

¹J/Q/K are treated as 10.

the candidate and prompt cards change. The correct action is to select the card whose suit matches the prompt card, with a confounder card sharing the same number but not the suit. The action set is $\{0, 1, 2, 3\}$, indexing the four candidates. To succeed, the agent must make $n_{\text{decisions}} = 5$ correct selections in a row, receiving reward +1 only on the final step, with incorrect selections ending the episode immediately with reward 0. CardMaze is a logic puzzle with many more possible states than EZPoints, so it is well suited to test SAGE’s generalization capabilities.

ALFWorld. ALFWorld (Shridhar et al., 2021) is a visually grounded, goal-conditioned household simulation benchmark that integrates a text-based interactive environment with a realistic household simulator, with each episode belonging to one of six task types. The agent observes a rendered scene and issues text-based actions (e.g., `go to counter 1, pick up mug 1`) from the admissible action set. This is state-dependent: only actions admissible in the current state are allowed, and the teacher prompt includes this admissible set. Episodes terminate upon task completion or when reaching an action limit, and reward is given as the ALFWorld goal-conditioned success rate (also rewarding partial completions)².

4.2. Baselines

As a reference point, we consider a **PPO** agent trained directly on visual observations without any external guidance, which establishes a lower bound on performance, due to the learning capacity of the RL agent. To also characterize the standalone capabilities of the VLM teacher, we also evaluate a **VLM-as-policy** baseline in which the VLM directly selects actions from image observations using CoT prompting.

We then compare against representative methods that integrate VLMs into RL. First, **RL-VLM-F** (Wang et al., 2024) leverages VLMs for reward shaping, by training a learned reward model from preference over state pairs generated by the VLM at fixed timestep intervals. Second, **LVLM2P** (Lee et al., 2025), which distills VLM predictions into the policy via supervised learning. In practice, this differs from SAGE in two ways: (1) the VLM is prompted at every timestep, and (2) the VLM generates a numerical probability distribution over actions, rather than a single optimal action.

4.3. SAGE Variants

To isolate the effects of selective guidance and behavior cloning, we evaluate several variants of our method. A guidance-only (**SAGE w/o BC**) variant queries the teacher

²As ALFWorld is partially observable, we add an LSTM module to the policy, whose hidden state is used for action generation, and a GRU text embedding for goals and admissible actions.

Method	FrozenLake	EZPoints	CardMaze	AlfWorld
PPO	0.01 $\pm$ 0.01	0.00 $\pm$ 0.00	0.01 $\pm$ 0.01	0.10 $\pm$ 0.00
VLM-as-policy	0.00 $\pm$ 0.01	-3.40 $\pm$ 0.07	0.67 $\pm$ 0.03	0.11 $\pm$ 0.00
RL-VLM-F	0.01 $\pm$ 0.02	-1.00 $\pm$ 0.00	0.09 $\pm$ 0.02	–
LVLM2P	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.01 $\pm$ 0.01	–
SAGE (w/o BC)	0.00 $\pm$ 0.01	-1.92 $\pm$ 2.28	0.04 $\pm$ 0.02	–
SAGE	0.26 $\pm$ 0.27	4.67 $\pm$ 1.89	0.99 $\pm$ 0.02	0.08 $\pm$ 0.02
Oracle (w/o BC)	0.07 $\pm$ 0.07	8.77 $\pm$ 0.91	0.11 $\pm$ 0.05	0.12 $\pm$ 0.00
Oracle (w/ BC)	0.87 $\pm$ 0.05	10.00 $\pm$ 0.00	0.98 $\pm$ 0.04	0.10 $\pm$ 0.00

Table 1. Evaluation performance comparison across environments. Results report the peak episodic return (mean $\pm$ std. over three random seeds).

when policy entropy exceeds a threshold and executes the returned action without applying an auxiliary behavior cloning loss, similar to Merler et al. (2025). Our full **SAGE** method augments this with behavior cloning on guided timesteps, encouraging the policy to internalize useful teacher behavior while gradually regaining autonomy as confidence increases. Finally, an **Oracle**-guided variant replaces the VLM teacher with an oracle providing optimal rule-based actions, showing an upper-bound on SAGE’s performance independent of VLM accuracy, both with and without behavior cloning.

5. Results

We organize our experiments around four Research Questions (RQs) aimed at investigating how our method improves over the baselines, the impact of guidance during training and the impact of hyper-parameters. For all experiments, we use Gemma3-27B (Gemma Team et al., 2025) as the VLM of choice. We report our prompts and hyperparameters in Appendices A and B.

5.1. RQ1: Does SAGE improve performance over the baselines?

Figure 3 shows training return curves across all environments. SAGE exhibits fast learning, particularly in CardMaze and EZPoints, where performance improves rapidly despite sparse rewards, thanks to the use of VLM guidance. We additionally report evaluation return curves in Figure 4 to assess generalization of the learned policy, and show the maximum performance in evaluation for all methods in Table 1. Unlike training, evaluation is performed without VLM guidance and uses distinct environment seeds; performance therefore reflects only what has been internalized by the RL policy. On all environments, with the exception of ALFWorld, SAGE meaningfully improves compared to both PPO and VLM-as-policy, demonstrating the mutually beneficial relationship of the two components in our method.

In particular, SAGE achieves almost 47% of the maximum achievable evaluation return on EZPoints, and reaches near-optimal performance on CardMaze after 55k steps Figure 4

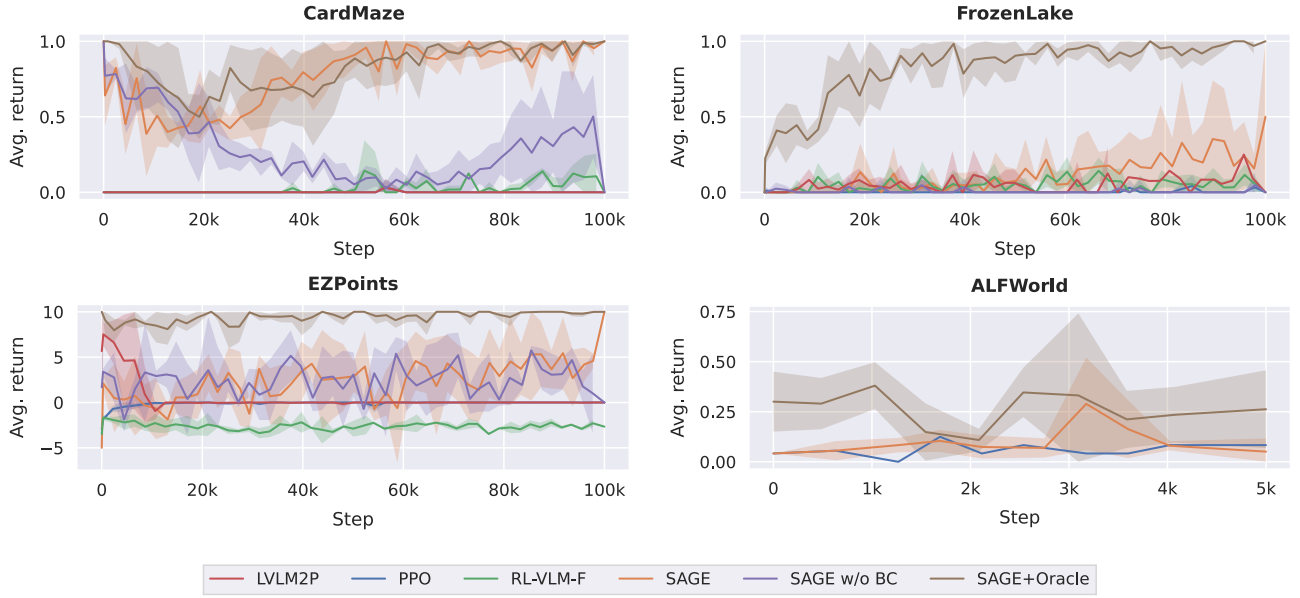


Figure 3. **Training performance across environments.** We report the mean training return across three random seeds, with standard deviation shown in the shaded area.

showing its sample efficiency. In FrozenLake the average return still increasing after 100k steps and reaches around 26%. Frozenlake remains a challenging environment due to the inability of Gemma3-27B to reliably infer the grid structure and the agent’s position in the grid-world, resulting in poor guidance (see examples in Appendix C), as also shown by the VLM-as-policy baseline.

Among the baselines, RL-VLM-F struggles to achieve consistent gains, as reward shaping is ineffective when visual states differ only subtly, making it difficult to assign informative preferences. Prior successes of RL-VLM-F and LVL2M2P rely on significantly stronger closed-source VLMs (Gemini Pro (Gemini Team et al., 2023) and Gemini 1.5 (Gemini Team et al., 2024) respectively), whereas SAGE remains effective even with a comparatively weak teacher. Nonetheless, RL-VLM-F improves over PPO in CardMaze and FrozenLake. LVL2M2P fails to learn in all environments; qualitative inspection suggests that Gemma3-27B cannot reliably produce coherent action probability distributions in plain text, which is a core requirement of the method.

For ALFWorld, we report results only for SAGE, PPO and VLM-as-policy under a reduced budget of 5k steps, due to the combined time cost of the simulator and VLM inference. Baseline methods weren’t tested: LVL2M2P was unstable with invalid probability outputs, and RL-VLM-F couldn’t be meaningfully explored within the available budget. Oracle runs use the handcoded expert actions provided by ALFWorld, which do not guarantee perfect behavior.

5.2. RQ2: How does the quality of guidance and behavior cloning impact the performance?

Oracle Guidance. Replacing the VLM with an oracle teacher confirms that SAGE scales effectively with guidance quality, improving performance in FrozenLake (0.26 $\rightarrow$ 0.87, as shown in Table 1) and EZPoints (4.67 $\rightarrow$ 10.00), confirming that failures stem from VLM grounding issues rather than the framework itself. However, Figures 3 and 4 show that SAGE with oracle guidance is comparable to the VLM guidance in CardMaze, even if it eventually converges on optimal performance. We hypothesize that, while the oracle provides perfect signals, its deterministic nature may cause the agent to overfit to specific trajectories; conversely, the VLM’s suboptimal stochasticity acts as a regularizer and avoids value overestimation.

Behavior Cloning. Ablating the behavior cloning objective reveals it to be the primary driver of policy internalization. SAGE without BC fails to achieve above-zero performance in all environments, despite receiving guidance from the VLM (Table 1). Even with a perfect oracle teacher, removing BC degrades returns by 88.78% in CardMaze and 91.95% in FrozenLake. This confirms that using guidance only as off-policy exploration is insufficient; without the explicit supervision of $\mathcal{L}_{BC}$, the PPO attenuates the gradients from guided actions, preventing the policy from internalizing the teacher’s behavior.

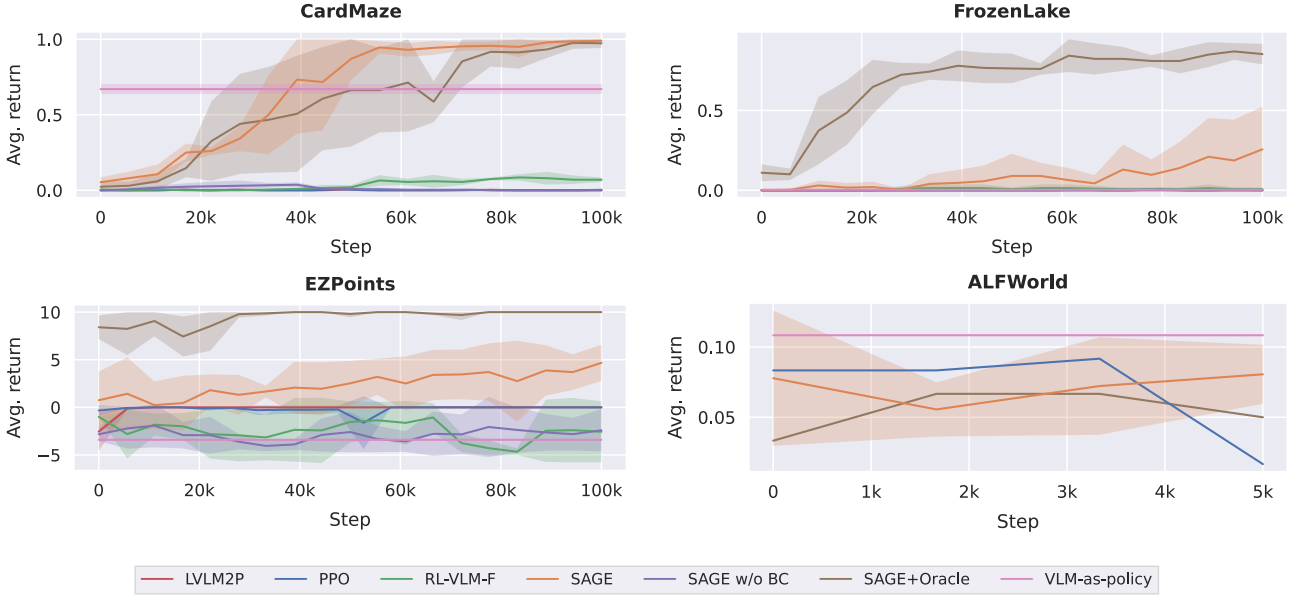


Figure 4. **Evaluation performance across environments.** We report the mean training return across three random seeds, with standard deviation shown in the shaded area.

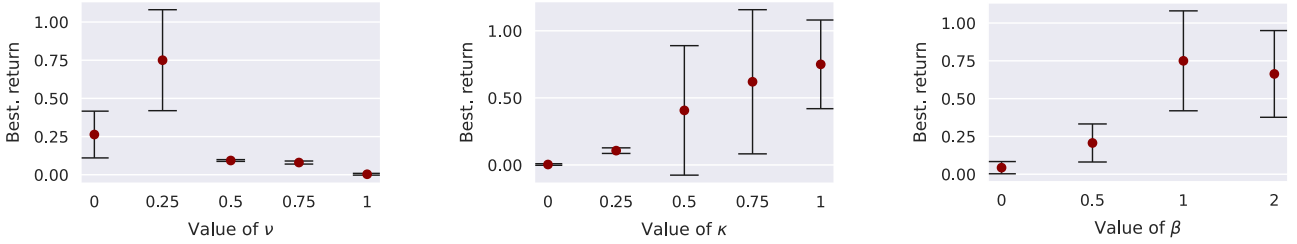


Figure 5. **Sensitivity Study.** We conduct a sensitivity analysis of the ν , κ , and β hyperparameters on the CardMaze environment. Starting from a base configuration of $\nu = 0.25$, $\kappa = 1$, and $\beta = 1$, we vary each hyperparameter independently while keeping the others fixed.

5.3. RQ3: How is SAGE’s performance affected by the hyperparameters?

We perform a sensitivity analysis on the hyperparameters, varying the entropy threshold ν , stochastic gating κ , and BC coefficient β . We choose CardMaze as a representative environment, and report the results in Figure 5. With too low values of ν , such as 0, the method overfits on the VLM guidance, unable to explore through RL and thus dropping in performance. Conversely, large ν values produce too few guidance calls, limiting the benefits of the teacher.

Similarly, performance increases smoothly with increased values of κ , directly correlating return with how often guidance is requested. This also shows that $\kappa = 1$ is the optimal value for this parameter, which we hypothesize is due to imperfect VLM suggestions already acting as a regularizer (as discussed in Section 5.2). Finally, larger β values increase performance thanks to improved behavior cloning strength;

however, too-large values cause overfitting to teacher errors, with the optimal value being 1 as a trade-off.

5.4. RQ4: How much and when is guidance actually used?

We measure the percentage of actions which exceed ν during training (Figure 6) and the number of actual VLM calls (Figure 7) for FrozenLake, EZPoints, and CardMaze.

Across all environments, guidance is concentrated in early training and decreases sharply as the policy entropy falls below ν . This is a consequence of the entropy-gated design: guidance is primarily requested during phases of high uncertainty, effectively bootstrapping the agent with useful VLM biases, and naturally diminishes as the policy distribution becomes sharper, thanks in part to behavior cloning.

The rate at which guidance usage declines varies across environments and reflects differences in task structure rather

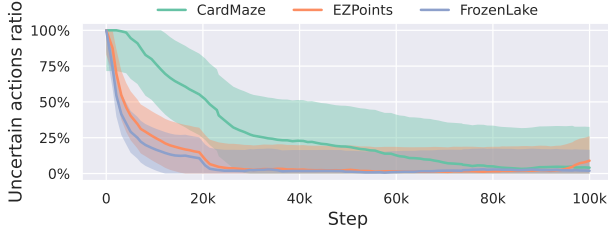


Figure 6. **High Entropy Actions.** We show the percentage of times the entropy exceeds ν during training, smoothed for readability.

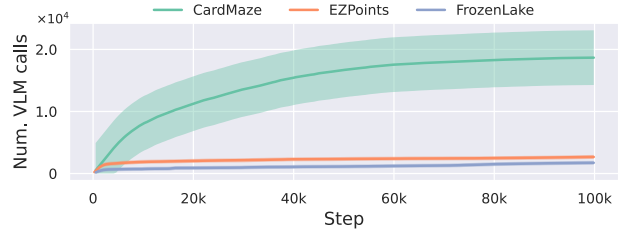


Figure 7. **Number of VLM prompts.** That is, this is the number of vlm calls after the cache.

than learning success alone. In FrozenLake, policy entropy drops rapidly (Figure 6) despite poor performance, hinting that the agent quickly becomes confident in suboptimal behaviors caused by poor VLM guidance. Similarly, in EZPoints, guidance also diminishes quickly, but in this case, it accompanies high performance, suggesting the agent has successfully internalized the teacher’s capabilities. In contrast, for CardMaze, guidance remains useful for longer, as the task involves symbolic reasoning and longer-horizon dependencies that sustain uncertainty during training.

The total number of VLM calls remains manageable due to selective triggering and caching. As shown in Figure 7, approximately 1.5k and 2.5k unique VLM queries are enough for training in FrozenLake and EZPoints, respectively. CardMaze, which presents greater state diversity, requires a larger number of queries (approximately 15k) to reach peak performance, but still well below the 100k total timesteps.

6. Conclusions

We introduce SAGE, a hybrid RL framework that incorporates a VLM as a temporary teacher. By prompting the VLM only when the agent’s policy exhibits high uncertainty and distilling the resulting guidance through a behavior cloning objective, SAGE enables a model-free RL policy to internalize useful priors while retaining the ability to learn through interaction. Across four different environments, including our novel contribution CardMaze, SAGE consistently outperforms PPO, the VLM teacher policy, and competitive VLM+RL baselines. Since evaluation is performed without any guidance, this shows that the policy generalizes and improves upon the guidance it receives during training. Moreover, oracle results indicate that the full potential of SAGE is primarily bounded by the quality of the teacher, rather than by the method itself. We show that SAGE effectively bootstraps the agent during early training, with guidance naturally fading as policy confidence increases. In principle, if the agent encounters unfamiliar states, the teacher can still be queried again. However, in practice, only a small number of VLM calls (1.5k–15k over 100k steps) are sufficient, making SAGE far more computationally efficient than methods that rely on constant VLM interaction.

6.1. Limitations & Future Work

While SAGE demonstrates strong empirical performance, some limitations suggest promising directions for future research. First, the method relies on policy entropy as a proxy for uncertainty when deciding whether to request guidance. Although effective and easy to compute, entropy may conflate genuine uncertainty with action multimodality. Exploring more uncertainty estimates, such as ensemble disagreement or the deviation between advantages, could enable more precise and efficient guidance.

Second, the effectiveness of SAGE is inherently limited by the quality of the guiding VLM. When the model struggles to accurately interpret visual details, as in FrozenLake, performance can struggle. In practice, while our method is VLM-agnostic, and stronger models can be used without any changes, inherent issues with current VLMs still persist. In particular, our experiments are focused on discrete action spaces: extending the framework to continuous control remains challenging, as VLMs struggle to generate precise numerical actions. Similarly, our experiments in ALFWorld are limited due to computational requirements, and could be further expanded.

Looking ahead, several extensions are particularly promising. Rather than relying on fixed uncertainty thresholds, future work could learn when to request guidance online, allowing the agent to adapt its reliance on the teacher over the course of training, based on the difficulty of the task. Additionally, closing the loop between agent and teacher, by using interaction data to adapt or fine-tune the guiding VLM, could further improve the quality of guidance. Together, these directions point toward more adaptive and mutually improving systems for efficient RL.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none which we feel must be specifically highlighted here.

References

- Abbeel, P. and Ng, A. Y. Apprenticeship learning via inverse reinforcement learning. In *Proceedings of the twenty-first international conference on Machine learning*, pp. 1, 2004.
- Ahn, M., Ichter, B., Brohan, A., Chebotar, Y., Finn, C., Hausman, K., et al. Do as i can, not as i say: Grounding language in robotic affordances. In *6th Annual Conference on Robot Learning*, 2022.
- Bommasani, R., Hudson, D. A., Adeli, E., Altman, R., Arora, S., von Arx, S., Bernstein, M. S., Bohg, J., Bosselut, A., Brunskill, E., et al. On the opportunities and risks of foundation models. *arXiv preprint arXiv:2108.07258*, 2021.
- Bonetta, G., Zago, D., Cancelliere, R., Polato, M., and Magnini, B. Vision language models as policy learners in reinforcement learning environments. In *ESANN*, 2024.
- Brohan, A., Brown, N., Carbajal, J., Chebotar, Y., Chen, X., Choromanski, K., Ding, T., Driess, D., Dubey, A., Finn, C., et al. Rt-2: Vision-language-action models transfer web knowledge to robotic control. *arXiv preprint arXiv:2307.15818*, 2023.
- Burda, Y., Edwards, H., Storkey, A., and Klimov, O. Exploration by random network distillation. In *International Conference on Learning Representations*, 2019.
- Cao, Y., Zhao, H., Cheng, Y., Shu, T., Chen, Y., Liu, G., Liang, G., Zhao, J., Yan, J., and Li, Y. Survey on large language model-enhanced reinforcement learning: Concept, taxonomy, and methods. *IEEE Transactions on Neural Networks and Learning Systems*, 2024.
- Carta, T., Romac, C., Wolf, T., Lamprier, S., Sigaud, O., and Oudeyer, P.-Y. Grounding large language models in interactive environments with online reinforcement learning. In *Proceedings of the 40th International Conference on Machine Learning, ICML’23. JMLR.org*, 2023.
- Da Silva, F. L., Hernandez-Leal, P., Kartal, B., and Taylor, M. E. Uncertainty-aware action advising for deep reinforcement learning agents. *Proceedings of the AAAI Conference on Artificial Intelligence*, 34(04):5792–5799, Apr. 2020.
- Du, Y., Watkins, O., Wang, Z., Colas, C., Darrell, T., Abbeel, P., Gupta, A., and Andreas, J. Guiding pretraining in reinforcement learning with large language models. In *International Conference on Machine Learning*, pp. 8657–8677. PMLR, 2023.
- Gemini Team, Anil, R., Borgeaud, S., Alayrac, J.-B., Yu, J., Soricut, R., Schalkwyk, J., Dai, A. M., Hauth, A., Millican, K., et al. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*, 2023.
- Gemini Team, Georgiev, P., Lei, V. I., Burnell, R., Bai, L., Gulati, A., Tanzer, G., Vincent, D., Pan, Z., Wang, S., et al. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. *arXiv preprint arXiv:2403.05530*, 2024.
- Gemma Team, Kamath, A., Ferret, J., Pathak, S., Vieillard, N., Merhej, R., Perrin, S., Matejovicova, T., Ramé, A., Rivière, M., et al. Gemma 3 technical report. *arXiv preprint arXiv:2503.19786*, 2025.
- Huang, W., Abbeel, P., Pathak, D., and Mordatch, I. Language models as zero-shot planners: Extracting actionable knowledge for embodied agents. In *International conference on machine learning*, pp. 9118–9147. PMLR, 2022a.
- Huang, W., Xia, F., Xiao, T., Chan, H., Liang, J., Florence, P., Zeng, A., Tompson, J., Mordatch, I., Chebotar, Y., Sermanet, P., Jackson, T., Brown, N., Luu, L., Levine, S., Hausman, K., and brian ichter. Inner monologue: Embodied reasoning through planning with language models. In *6th Annual Conference on Robot Learning*, 2022b.
- Kahneman, D. *Thinking, Fast and Slow*. Allen Lane, 2011. ISBN 9781846140556.
- Kim, M. J., Pertsch, K., Karamcheti, S., Xiao, T., Balakrishna, A., Nair, S., Rafailov, R., Foster, E., Lam, G., Sanke, P., et al. Openvla: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.
- Klissarov, M., D’Oro, P., Sodhani, S., Raileanu, R., Bacon, P.-L., Vincent, P., Zhang, A., and Henaff, M. Motif: Intrinsic motivation from artificial intelligence feedback. In *NeurIPS 2023 Foundation Models for Decision Making Workshop*, 2023.
- Knox, W. B. and Stone, P. Interactively shaping agents via human reinforcement: The tamer framework. In *Proceedings of the fifth international conference on Knowledge capture*, pp. 9–16, 2009.
- Kojima, T., Gu, S. S., Reid, M., Matsuo, Y., and Iwasawa, Y. Large language models are zero-shot reasoners. *Advances in neural information processing systems*, 35: 22199–22213, 2022.
- Kwon, M., Xie, S. M., Bullard, K., and Sadigh, D. Reward design with language models. In *The Eleventh International Conference on Learning Representations*, 2023.
- Lambert, N. Reinforcement learning from human feedback. *arXiv preprint arXiv:2504.12501*, 2025.

- Laud, A. D. *Theory and application of reward shaping in reinforcement learning*. PhD thesis, University of Illinois at Urbana-Champaign, 2004.
- Lee, D., Luu, T. M., Lee, Y., and Yoo, C. D. Sample efficient reinforcement learning via large vision language model distillation. In *ICASSP 2025-2025 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pp. 1–5. IEEE, 2025.
- Liang, J., Huang, W., Xia, F., Xu, P., Hausman, K., Ichter, B., Florence, P., and Zeng, A. Code as policies: Language model programs for embodied control. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 9493–9500. IEEE, 2023.
- Liu, H., Li, C., Wu, Q., and Lee, Y. J. Visual instruction tuning. *Advances in neural information processing systems*, 36:34892–34916, 2023.
- Ma, Y. J., Sodhani, S., Jayaraman, D., Bastani, O., Kumar, V., and Zhang, A. VIP: Towards universal visual reward and representation via value-implicit pre-training. In *NeurIPS 2022 Foundation Models for Decision Making Workshop*, 2022.
- Ma, Y. J., Kumar, V., Zhang, A., Bastani, O., and Jayaraman, D. LIV: Language-image representations and rewards for robotic control. In *Workshop on Reincarnating Reinforcement Learning at ICLR 2023*, 2023a.
- Ma, Y. J., Liang, W., Wang, G., Huang, D.-A., Bastani, O., Jayaraman, D., Zhu, Y., Fan, L., and Anandkumar, A. Eureka: Human-level reward design via coding large language models. *arXiv preprint arXiv:2310.12931*, 2023b.
- Merler, M., Bonetta, G., and Magnini, B. Guiding reinforcement learning with selective vision-language model supervision. In Niggemann, O., Biswas, G., Micheli, A., Heesch, R., Diedrich, A., Ehrhardt, J., and Widulle, N. (eds.), *ECAI 2025 Workshop on AI-based Planning for Complex Real-World Applications (CAIPI’25)*, pp. 39–51, Bologna, Italy, October 2025. CEUR Workshop Proceedings.
- Ng, A. Y., Harada, D., and Russell, S. J. Policy invariance under reward transformations: Theory and application to reward shaping. In *Proceedings of the Sixteenth International Conference on Machine Learning*, pp. 278–287, 1999.
- Parisotto, E., Ba, J., and Salakhutdinov, R. Actor-mimic: Deep multitask and transfer reinforcement learning. In *ICLR*, 2016.
- Rocamonde, J., Montesinos, V., Nava, E., Perez, E., and Lindner, D. Vision-language models are zero-shot reward models for reinforcement learning. In *The Twelfth International Conference on Learning Representations*, 2024.
- Ross, S., Gordon, G., and Bagnell, D. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the fourteenth international conference on artificial intelligence and statistics*, pp. 627–635. JMLR Workshop and Conference Proceedings, 2011.
- Rusu, A. A., Colmenarejo, S. G., Gulcehre, C., Desjardins, G., Kirkpatrick, J., Pascanu, R., Mnih, V., Kavukcuoglu, K., and Hadsell, R. Policy distillation. *arXiv preprint arXiv:1511.06295*, 2015.
- Schaal, S. Learning from demonstration. In Mozer, M., Jordan, M., and Petsche, T. (eds.), *Advances in Neural Information Processing Systems*, volume 9. MIT Press, 1996.
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- Shridhar, M., Yuan, X., Côté, M.-A., Bisk, Y., Trischler, A., and Hausknecht, M. ALFWorld: Aligning Text and Embodied Environments for Interactive Learning. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021.
- Sutton, R. S., Barto, A. G., et al. *Reinforcement learning: An introduction*. MIT press Cambridge, 1998.
- Tan, W., Zhang, W., Liu, S., Zheng, L., Wang, X., and An, B. True knowledge comes from practice: Aligning large language models with embodied environments via reinforcement learning. In *The Twelfth International Conference on Learning Representations*, 2024.
- Towers, M., Kwiatkowski, A., Terry, J., Balis, J. U., De Cola, G., Deleu, T., Goulão, M., Kallinteris, A., Krimmel, M., KG, A., et al. Gymnasium: A standard interface for reinforcement learning environments. *arXiv preprint arXiv:2407.17032*, 2024.
- Venuto, D., Islam, S. N., Klissarov, M., Precup, D., Yang, S., and Anand, A. Code as reward: empowering reinforcement learning with vlms. In *Proceedings of the 41st International Conference on Machine Learning, ICML’24*. JMLR.org, 2024.
- Wang, Y., Sun, Z., Zhang, J., Xian, Z., Biyik, E., Held, D., and Erickson, Z. RL-vlm-f: Reinforcement learning from vision language foundation model feedback. In *Proceedings of the 41th International Conference on Machine Learning*, 2024.

- Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., Le, Q. V., Zhou, D., et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.
- Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Gao, C., Huang, C., Lv, C., et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- Ye, W., Zhang, Y., Weng, H., Gu, X., Wang, S., Zhang, T., Wang, M., Abbeel, P., and Gao, Y. Reinforcement learning with foundation priors: Let embodied agent efficiently learn on its own. In *8th Annual Conference on Robot Learning*, 2024.
- Zhai, Y., Bai, H., Lin, Z., Pan, J., Tong, S., Zhou, Y., Suhr, A., Xie, S., LeCun, Y., Ma, Y., and Levine, S. Fine-tuning large vision-language models as decision-making agents via reinforcement learning. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024.
- Zhou, Z., Hu, B., Zhao, C., Zhang, P., and Liu, B. Large language model as a policy teacher for training reinforcement learning agents. *IJCAI ’24*, 2024.

A. VLM Guidance Prompts

We present the few-shot prompts with CoT used to query the VLM to guide the SAGE agent. The {image}, {mission} and {admissible_commands} fields are replaced at runtime with the state s_t and corresponding information.

Frozenlake

<user>

You are a smart agent acting in the "FrozenLake" environment from the Gymnasium library. You will receive observations from the environment and must decide which action to take next.

The goal is to navigate across a frozen grid to reach the goal without falling into holes.

Observation space

You are presented with the image of the environment in the style of pixel art.

Each cell may be safe (white with snow), a hole (bright blue ice hole), the start (a stool) or the goal (a present box). The agent is represented by an elf character.

Action space

The action space includes 4 discrete actions:

- 0: Move left
- 1: Move down
- 2: Move right
- 3: Move up

Output Format

You should think about the answer step by step inside the <think> </think> tags, then provide the chosen action index inside the <action> </action> tags. All text outside the <think> and <action> tags will be ignored.

Few-shot Examples (2-shot)

Example 1

Observation:

image: [the image shows the elf in row 4, column 7 (i.e. the coordinates are from 1 to 8). There is a hole left of him and one below.]

<think>

The agent is currently at the center right corner of the grid. The goal is below the agent. The agent needs to avoid the hole and then proceed downwards toward the goal which is located at the bottom right corner. The possible moves to avoid the holes are "move up" and "move right". Moving up is safe but would result in the agent getting further away from the gift. Given all these considerations probably a good option is going up and reevaluate the situation, and a even better option is to go right. Move right is action 2.

</think>

<action>2</action>

Example 2

Observation:

image: [the image shows the elf in row 3, column 3. There is a hole directly to the right, safe tiles straight down toward the bottom row, and the goal is further down to the right.]

<think>

The elf is a couple of tiles above the center of the grid. The cell to the right is a hole, so moving right (action 2) is risky. Moving left would take the agent back toward the start and does not progress toward the goal. Moving up only keeps the elf wandering on the same plateau. The only safe and progress-making move is to step down along the clear column toward the goal. Therefore, action 1 (move down) is preferred.

</think>

<action>1</action>

Current Task

History:

Current Observation:

{image}

Based on the current visual information, which action should the agent take next?

</user>

Ezpoints

<user>

You are playing a card game. The goal is to output a math formula that evaluates to 12 using the numbers in the poker cards. The goal is to use all numbers in the cards (only once) to compute 12. At each state, only numbers that appear in the cards and math operators are legal actions. "J", "Q", or "K" are treated as "10", and ace cards are treated as "1". The game ends when the "=" operator is added to the formula.

Workflow Checklist

- Begin each turn with a concise checklist: (1) analyze your current hand and the displayed formula, (2) identify all valid actions, (3) select the optimal move that advances towards a valid formula evaluating to 12, (4) output the corresponding action number.

Observation space

You will receive an image displaying:

- The two cards currently in hand.
- The current, possibly incomplete formula (displayed after 'Formula:').

Action space

The action space includes 13 discrete actions:

- 1: add number "1" to the formula
- 2: add number "2" to the formula
- 3: add number "3" to the formula
- 4: add number "4" to the formula
- 5: add number "5" to the formula
- 6: add number "6" to the formula
- 7: add number "7" to the formula
- 8: add number "8" to the formula
- 9: add number "9" to the formula
- 10: add number "10" to the formula
- 11: Add the operator "+" (addition) to the formula
- 12: Add the operator "*" (multiplication) to the formula
- 13: Add the operator "=" (equals) to the formula

Output Format

You should think about the answer step by step inside the <think> </think> tags, then provide the chosen action index within the <action> </action> tags. All text outside the <think> and <action> tags will be ignored.

Few-shot Examples

Example 1

Observation:

image: [the image shows two cards in hand: "J of clubs" and "2 of spades"; Formula: "" (empty)]

<think>

The cards in hand are a 2 of spades and a Jack of clubs. Since Jack is treated as 10, the numbers available are 2 and 10. The goal is to create a formula that evaluates to 12.

Possible actions:

1. Add the number 2 to the formula.
2. Add the number 10 to the formula.
3. Add the addition operator (+).
4. Add the multiplication operator (*).
5. Add the equals operator (=) if a valid formula is formed.

Starting with the numbers:
 – 2 and 10 can be combined with addition: $2 + 10 = 12$.

The simplest formula is $2 + 10 = 12$. Also $10 + 2 = 12$ is valid.
 Let's go with the first option, so add the number 2 to the formula.
 Number 2 corresponds to action 2.
 </think>
 <action>2</action>

Example 2
 ## Observation:
 image: [the image shows two cards in hand: "7 of diamonds" and "5 of diamonds"; Formula: "10"]

<think>
 Okay, let's see. The cards I have are a 7 of diamonds and a 5 of diamonds. The current formula is "7". I need to use both numbers to make the formula equal 12.

So far, I have "7". If I add 5 next, that would make it $7 + 5$, which equals 12. That works! I should add the 5 to the formula.
 To output add "5" to the formula i need to output the "+" symbol first. The "+" symbol corresponds to action 11.
 </think>
 <action>11</action>

Example 3
 ## Observation:
 image: [the image shows two cards in hand: "Q of hearts" and "2 of clubs"; Formula: "10"]

<think>
 The cards in hand are the Queen of hearts and the 2 of clubs. The Queen is treated as 10. The current formula is "10".

Possible actions:
 1. Add the number 10 to the formula.
 2. Add the number 2 to the formula.
 3. Add the addition operator (+).
 4. Add the multiplication operator (*).
 5. Add the equals operator (=).

Since the current formula is "10", adding the number 2 will give $10 + 2$.

$10 + 2 = 12$, which is the target value.

So, since the formula is just "10", the next action should be to add the operator "+" to the formula.
 The + operator corresponds to action index 11.
 </think>
 <action>11</action>

Current Task
 ## History:
 ## Current Observation:
 {image}

Based on the current visual information, which action should the agent take next?
 </user>

Cardmaze

<user>
 You are an expert agent playing a simple game of cards.
 You will be presented with {n_cards} different poker cards at the top of the image. At the bottom of the image, you will be shown a larger poker card which acts as your prompt.
 You will be given a task, which determines which card you should select from the top cards, based on the card shown in the prompt.

If you make a mistake, the progress will reset to zero and you will have to start over. The game ends when you have made { n_decisions} correct actions in a row.

Workflow Checklist
 – Begin each turn with a concise checklist: (1) analyze the image to identify which card is in the prompt and the top cards, (2) find which card at the top matches the prompt according to the task, (3) identify the index of the matching card, (4) select the action which corresponds to the index of the matching card.

Output Format
 You should think about the answer step by step inside the <think> </think> tags, provide the chosen action index inside the <action> </action> tags. All text outside the <think> and <action> tags will be ignored.

Few-shot Examples

Example 1
Task
 Your task is to produce an action corresponding to the card at the top that matches the suit (hearts, diamonds, clubs, spades) of the prompt card, but not its number or face.

Action space
 The action space includes 4 discrete actions:
 – 0: Choose the 1st card from the left
 – 1: Choose the 2nd card from the left
 – 2: Choose the 3rd card from the left
 – 3: Choose the 4th card from the left

Observation:
 image: [the image shows 4 cards in hand (top of the image): "J of clubs", "2 of spades", "5 of diamonds" and "K of hearts"; The prompt card (image center): "3 of clubs"]

<think>
 The top cards are the J of clubs, the 2 of spades, the 5 of diamonds and the K of hearts. The big central card shows a 3 of clubs.

I have to choose from the top cards the one that matches the suit of the 3 of clubs. There is only one with the same suit, which is the J of clubs.
 The J of clubs is the first card from the left, so i choose action 0.

</think>
 <action>0</action>

Example 2
Task
 Your task is to produce an action corresponding to the card at the top that matches the suit (hearts, diamonds, clubs, spades) of the prompt card, but not its number or face.

Action space
 The action space includes 4 discrete actions:
 – 0: Choose the 1st card from the left
 – 1: Choose the 2nd card from the left
 – 2: Choose the 3rd card from the left
 – 3: Choose the 4th card from the left

Observation:
 image: [the image shows 4 cards in hand (top of the image): "2 of diamonds", "3 of diamonds", "5 of hearts" and "J of hearts"; The prompt card (image center): "ace of diamonds"]

<think>
 The top cards are the 2 of diamonds, 3 of diamonds, 5 of hearts and the J of hearts. The big central card shows an ace of diamonds.

I have to choose from the top cards the one that matches the suit of the ace. the 2 and 3 of diamonds both are possible choices since they have the suit of diamonds.
I'll pick the 3 of diamonds as final choice, which corresponds to the 2nd card from the left. So the action is 1.

</think>
<action>1</action>

Current Task
{rule}

Action space
{action_space}

Observation:
{image}

Based on the current visual information, which action should the agent take next?
</user>

AlfWorld

You are an autonomous intelligent agent tasked with solving household tasks in the ALFWorld environment. Your goal is to interpret the visual and textual information to navigate the environment, interact with objects, and complete the specified high-level task (e.g., "Put a hot apple in the fridge").

Workflow Checklist

Begin each turn with a concise checklist:

- (1) **Analyze the Goal**: Understand the high-level objective and any state changes required (heating, cooling, cleaning).
- (2) **Analyze the Observation**: Examine the current image and the text description to understand your location and what objects are visible or in your inventory.
- (3) **Evaluate Actions**: Review the list of "Admissible Actions" provided.
- (4) **Select Optimal Move**: Choose the action that logically advances the plan (e.g., if you need an apple and see it, pick it up; if you hold it and need to heat it, go to the microwave).
- (5) **Output Action**: Output the index corresponding to the chosen action.

Observation Space

You will receive:

1. **Image**: A visual frame of your current view in the simulated environment.
2. **Text Observation**: A natural language description of the current state or feedback from the previous action.
3. **Goal**: The natural language instruction describing the task to be completed.
4. **Admissible Actions**: A numbered list of valid text commands available in the current state.

Action Space

You must select **one** action from the provided list of "Admissible Actions".
Output the **index** (number) associated with that action.

Output Format

You should think about the answer step by step inside the <think> </think> tags to check your plan and reasoning. Then, provide the chosen action index within the <action> </action> tags. All text outside the tags will be ignored.

Few-shot Examples

Example 1

Observation:

Goal: "Put a clean apple in the fridge."

Text Observation: "You are in the middle of a room. Looking quickly around you, you see a countertop 1, a fridge 1, a sinkbasin 1, and a stoveburner 1."

Admissible Actions:

0: look

```

1: goto countertop 1
2: goto fridge 1
3: goto sinkbasin 1
4: goto stoveburner 1
**Image**: [Image shows a general view of a kitchen with various receptacles visible but no apple in immediate focus]

<think>
1. **Goal**: I need to find an apple, clean it, and then put it in the fridge.
2. **State**: I am in the middle of the room. I do not see an apple yet, but I see receptacles where one might be (countertop,
fridge, sinkbasin).
3. **Plan**: I need to explore the environment to find the apple. A countertop is a common place for fruit.
4. **Action**: I will go to the countertop to check for the apple. Action index 1 is "goto countertop 1".
</think>
<action>1</action>

## Example 2
## Observation:
**Goal**: "Put a clean apple in the fridge."
**Text Observation**: "You arrive at countertop 1. On the countertop 1, you see a apple 1, a bread 1, and a knife 1."
**Admissible Actions**:
0: look
1: goto fridge 1
2: goto sinkbasin 1
3: pickup apple 1
4: pickup bread 1
5: pickup knife 1
**Image**: [Image shows a close-up of a countertop with an apple, a loaf of bread, and a knife visible]

<think>
1. **Goal**: Clean an apple and put it in the fridge.
2. **State**: I am at the countertop and I see the apple I need. My inventory is likely empty or I have space.
3. **Plan**: The next logical step is to acquire the object required for the task.
4. **Action**: I should pick up the apple. Action index 3 is "pickup apple 1".
</think>
<action>3</action>

## Example 3
## Observation:
**Goal**: "Put a clean apple in the fridge."
**Text Observation**: "You pick up the apple 1 from the countertop 1."
**Admissible Actions**:
0: look
1: goto fridge 1
2: goto sinkbasin 1
3: goto stoveburner 1
4: put apple 1 on countertop 1
**Image**: [Image shows the agent holding an apple in front of the countertop]

<think>
1. **Goal**: Clean the apple, then fridge it.
2. **State**: I am holding the apple. It is currently dirty (implied, as I just picked it up and haven't cleaned it).
3. **Plan**: Before I can put it in the fridge, the goal specifies it must be "clean". I need to find a water source to clean it.
4. **Action**: I see "goto sinkbasin 1" in the list. This is the correct location to clean objects. Action index 2.
</think>
<action>2</action>

## Example 4
## Observation:
**Goal**: "Put a hot potato in the bin."
**Text Observation**: "You arrive at microwave 1. The microwave 1 is closed."
**Admissible Actions**:
0: look
1: goto garbagecan 1

```

```

2: open microwave 1
3: heat potato 1 in microwave 1
**Image**: [Image shows a closed microwave]

<think>
1. **Goal**: Heat a potato and put it in the bin.
2. **State**: I am holding the potato (implied by the ability to heat it) and I am in front of the microwave. The microwave is closed.
3. **Plan**: To heat the potato, I need to use the microwave. Usually, I might need to open it first, or if the "heat" command is available directly, I can use that.
4. **Action**: "heat potato 1 in microwave 1" (index 3) is explicitly available. Let's try that directly, and if it fails, I can open it next time.
</think>
<action>3</action>

```

```

-----

# Current Task
## Observation:
{mission}
**Admissible Actions**:
{admissible_commands}
**Image**: {image}

```

Based on the current visual and textual information, which action index should the agent take next?

B. Experiment Hyperparameters

We report the hyperparameters used for SAGE and PPO training in our experiments on all environments, in Tables 2 and 3 respectively.

Parameter	FrozenLake	EZPoints	CardMaze	ALFWorld
Guidance VLM	google/gemma-3-27b-it			
Kappa (κ)	1.0	1.0	1.0	1.0
Entropy threshold (ν)	0.75	0.25	0.25	0.25
Behavior Cloning coef (β)	1.0	1.0	1.0	1.0
BC schedule	linear			
Cache size	10000			

Table 2. SAGE hyperparameters across environments.

Parameter	Value
Agent type	CNN (+ LSTM text encoder for ALFWorld)
Convolution channels	[16, 32, 32]
Conv head out features	256
Actor std init	0.01
Critic std init	1
Gamma (γ)	0.99
GAE lambda (λ)	0.95
Value loss coef	0.5
Entropy coef	0.01
Clip coef	0.2
Clip value loss	true
Advantage normalization	true
Learning rate annealing	true
Batch size	512

Table 3. PPO training hyperparameters.

C. Qualitative Analysis and Failure Cases of VLM Guidance

We present some example failure cases for SAGE and LVLM2P, involving Gemma3-27B failing to generate the optimal action.

Examples A and B show sample responses from SAGE, with the VLM failing to properly understand the grid-world and assess spatial relationships. This could be due to the way the image is processed, or because pixel-art grid-worlds are not too prevalent in the model’s pre-training distribution, making it unable to properly reason over exact coordinates.

Example C instead shows a sample response from LVLM2P-style prompting on EZPoints, highlighting the difficulty for VLMs to produce exact state distributions, particularly as the number of actions grows. SAGE mitigates this by only asking for the optimal action, rather than a full distribution.

Example A - FrozenLake Failure Case



VLM response

```
<think>
The agent (elf) is currently in the middle of the
grid, close to the right edge. The goal
(present box) is in the bottom right corner.
There are holes surrounding the agent,
specifically below, to the left and to the
right.
Moving left (0) is immediately dangerous. Moving
down (1) is also dangerous. Moving right (2)
is also dangerous. Moving up (3) seems to be
the only safe option, as it avoids the holes.
</think>
<action>3</action>
```

Comment

The VLM is not able to assess the precise spatial relationships between the agent and the different holes.

Example B - FrozenLake Failure Case



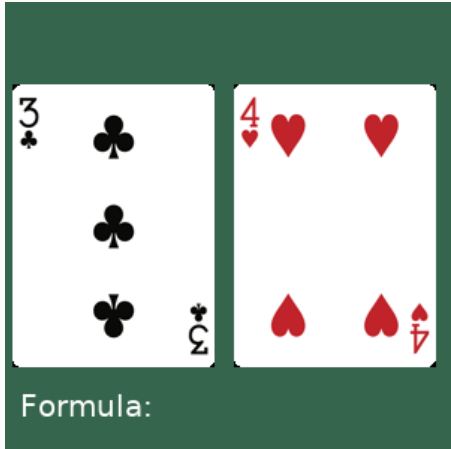
VLM response

```
<think>
The agent (elf) is currently in row 2, column 2.
The goal (present box) is in row 8, column 7.
There are holes around the agent. Moving down
(action 1) is a possibility, as the tile
below is safe. Moving right (action 2) would
lead the agent into a hole. Moving up (action
3) is less promising since it takes the agent
further from the goal. Moving left (action 0)
would also take the agent further away from
the goal. Given the holes surrounding the
agent, the best course of action is to go
down, where there's safe ground.
</think>
<action>1</action>
```

Comment

The VLM is confused about the hole positions and decides to fall in the only one nearby.

Example C - EZPoints Failure Case



VLM response

<think>

The cards in hand are a 3 of clubs and a 4 of hearts. The goal is to create a formula that evaluates to 12 using both numbers. The current formula is empty.

Possible actions:

1. Add the number 3 to the formula.
2. Add the number 4 to the formula.
3. Add the addition operator (+).
4. Add the multiplication operator (*).
5. Add the equals operator (=).

Let's explore some possibilities:

- $3 + 4 = 7$ (Not 12)
- $3 * 4 = 12$ (This works!)

So, the optimal strategy is to add 3, then add the multiplication operator, and then add 4. Or add 4, then the multiplication operator, then 3.

Let's start by adding the number 3 to the formula.

Number 3 corresponds to action index 2.

</think>

<action>2</action>

<probs>0.8, 0.0, 0.2, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0</probs>

Comment

The distribution of probability over actions is not coherent with the reasoning chain, since it gives just 0.2 probability for the chosen action and 0.8 probability to a non-valid action. This type of error is a problem for LVL2M2P, which heavily relies on precise fine-grained generation of probability distributions.